

CAF — Onboarding & External Context

Aggregated export — 2026-07-10. Source markdown in repo is unchanged.

Bundle id: `02-caf-onboarding-and-context`

Included: docs/EXTERNAL_CONTEXT_PACK.md

CAF Core — External context pack

Purpose: Tell another team, another repository, or an external LLM (ChatGPT, Claude, etc.) **everything it needs to understand, comment on, or re-implement CAF Core** — without pasting the full source tree.

How to use this file

1. Read this page first — it is the **table of contents and rules of engagement**.
2. Attach the **Tier 1 bundle** (below) to your LLM project or onboarding packet.
3. Add **Tier 2** docs when the topic is mimic, inputs, video, or publishing.
4. Attach **specific source files** only when asking for implementation-level changes.

Convention: Paths are from the **repository root**. Database schema is `caf_core`. The primary execution key is `task_id` scoped by `project_id`.

What CAF Core is (30-second version)

CAF (Content Automation Framework) is a **content automation pipeline**:

Signal Pack → **Candidates** → **Decision Engine** → **Content Jobs** → **LLM Drafts** → **QC** → **Rendering** → **Review** → **Publishing** → **Learning**

- **Source of truth:** PostgreSQL schema `caf_core`, especially `content_jobs` and `generation_payload`.
 - **Core API:** Fastify + TypeScript at repo root (`src/server.ts`).
 - **Review app:** Next.js client in `apps/review` — not the database of record.
 - **Media workers:** `services/renderer` (carousel PNGs), `services/video-assembly` (ffmpeg), `services/media-gateway` (combined port).
-

Tier 1 — Always include (general understanding)

Upload or paste these files together for **architecture, lifecycle, contracts, and invariants**:

#	File	What it gives the reader
0	docs/CAF_CURRENT_STATE_CONTEXT_PACK.md	Repo-derived current state (2026-07) — start here if context is stale
1	AGENTS.md	Invariants, “do not break,” where to change behavior
2	docs/CAF_CORE_COMPLETE_GUIDE.md	Single-file merged reference (~500 lines) — may lag BVS / new visual
3	docs/DOMAIN_MODEL.md	Entities, ID patterns, lifecycles (external copy of domain rules)
4	docs/ARCHITECTURE.md	Layer map, critical files, integration contracts
5	docs/LIFECYCLE.md	Run & job state machines
6	docs/TECH_STACK.md	Languages, services, third parties
7	docs/layers/README.md	Index to per-layer deep dives

ChatGPT PDF upload (4 volumes, everything operational): docs/export/pdf/11-caf-current-state-vol1-platform.pdf through 14-caf-current-state-vol4-ops.pdf — regenerate with `npm run export:doc-pdfs` . Source: docs/volumes/CAF_CONTEXT_VOL*.md .

Optional but recommended for rebuilds:

#	File	What it gives the reader
8	docs/REBUILD_FROM_DOCS.md	Step-by-step local/prod bootstrap
9	docs/DATABASE_SCHEMA.md	Table catalog grouped by domain
10	README.md	Quick start, CLI, deploy map
11	.env.example	Configuration surface (no secrets)
12	ENV_AND_SECRETS_INVENTORY.md	Full env var checklist

Tier 2 — Topic add-ons (attach when relevant)

Topic	Files
HTTP API examples	docs/API_REFERENCE.md, requests/caf-core.http
QC runtime	docs/QUALITY_CHECKS.md
Risk policies	docs/RISK_RULES.md

LLM prompt guidance	docs/GENERATION_GUIDANCE.md
Per-layer detail	docs/layers/*.md (10 files)
Video / HeyGen	docs/VIDEO_FLOWS.md, docs/HEYGEN_API_V3.md
Top-performer mimic	docs/MIMIC_FLOWS_COMPLETE_GUIDE.md, docs/MIMIC_IMAGE_FLOWS.md
Mimic text placement (future)	docs/MIMIC_TEXT_PLACEMENT_AUTOMATION.md
Creative Intelligence ingest	docs/CREATIVE_INTELLIGENCE.md
Inputs → signal pack	docs/CAF_INPUTS_PIPELINE_ROADMAP.md
Content job lifecycle	docs/JOB_LIFECYCLE.md
Production deploy	docs/FLY_PRODUCTION_CHECKLIST.md
Secrets safety	docs/USER_INPUT_AND_SECRETS.md
Product pitch	docs/CAF_PRODUCT_PITCH.md
Complete product guide	docs/CAF_COMPLETE_PRODUCT_GUIDE.md
Shareable PDFs by topic	docs/export/pdf/*.pdf — see docs/export/README.md
Doc reconciliation task	docs/CURSOR_DOC_RECONCILIATION_PROMPT.md — paste into Cursor to align stale docs with current-state pack
Stakeholder overview	docs/PROJECT_OVERVIEW.md

Tier 3 — Source code (implementation questions only)

Do **not** upload the entire `src/` tree. Attach **3–15 files** for the specific task.

Goal	Start here
Run planning	src/decision_engine/, src/services/run-orchestrator.ts
Job execution	src/services/job-pipeline.ts
LLM generation	src/services/llm-generator.ts, src/repositories/flow-engine.ts
QC	src/services/qc-runtime.ts
Mimic	src/services/mimic-draft-prep.ts, mimic-carousel-render.ts, src/domain/mimic-payload.ts

Review contract	src/routes/v1.ts, src/routes/review-contract.test.ts
Publications	src/routes/publications.ts
Learning	src/services/learning-rule-selection.ts, src/routes/learning.ts
Config / flags	src/config.ts
Schema truth	migrations/*.sql (relevant migrations only)

Scale reference (~2026): ~113k lines TypeScript in `src/` , ~23k in `apps/review` , ~70 SQL migrations, ~140 test files.

System prompt template (copy into ChatGPT / Custom GPT)

You are helping with CAF Core – a Fastify + PostgreSQL content automation platform.

Pipeline funnel:

Signal Pack → Candidates → Decision Engine → Content Jobs → Drafts → QC → Render → Review → Publish → Learn

Rules:

- `task_id` + `project_id` are the primary keys for jobs, drafts, assets, reviews.
- `content_jobs.generation_payload` is the main JSON contract – treat changes as API design.
- The Review app (`apps/review`) is a client; Postgres `caf_core` is source of truth.
- Do not propose renaming `task_id`, `run_id` patterns, or lifecycle enums without explicit approval.
- QC writes `qc_result` only via `mergeGenerationPayloadQc` (`generation-payload-qc.ts`).
- HeyGen retries must use `hasActiveProviderSession` (`content-job-render-state.ts`).
- Learning lookups go through `learning-rule-selection.ts` facade.
- `risk_policies` + brand `banned_words` are enforced by QC; project `risk_rules` are NOT (see `risk-qc-status`).

When suggesting code: name exact file paths. When unsure which layer is

affected, ask.
Prefer small, reviewable changes over repo-wide refactors.

Then attach **Tier 1** files from the table above.

What external readers can and cannot do from docs alone

Feasible from Tier 1 + 2	Requires source + tests
Explain architecture and workflow	Line-accurate refactors
Design a feature in the right layer	Prove dead-code removal is safe
Review API / contract changes	Re-implement every edge case
Bootstrap a new environment	Match 100% of admin UI behavior
Compare to another system	Replace LLM prompts verbatim

Honesty: docs/CAF_CORE_COMPLETE_GUIDE.md is a merged summary.
migrations/ and src/ win on conflicts.

Repository map (one screen)

```
CAF-Core/
├── src/                                # Core API (Fastify)
│   ├── server.ts                      # Entry
│   ├── config.ts                     # Env (Zod)
│   ├── routes/                       # HTTP (v1, runs, pipeline, admin, ...)
│   └── services/                     # Business logic (job-pipeline, llm-
generator, ...)
│   ├── decision_engine/              # Planning / scoring
│   ├── domain/                      # Typed payload readers, mimic contracts
│   └── repositories/                 # Postgres accessors
├── apps/review/                      # Next.js operator UI
├── services/
│   ├── renderer/                    # Carousel PNG worker
│   ├── video-assembly/              # ffmpeg worker
│   └── media-gateway/                # Combined gateway
├── migrations/                      # caf_core schema (versioned SQL)
├── docs/                             # All documentation
└── AGENTS.md                         # AI assistant invariants
```

```
|— .env.example          # Config template
|— ENV_AND_SECRETS_INVENTORY.md
```

Related docs

Doc	Audience
REBUILD FROM DOCS.md	Engineers bootstrapping from scratch
DATABASE SCHEMA.md	DB tables and relationships
DOMAIN MODEL.md	IDs and lifecycles
CAF CORE COMPLETE GUIDE.md	Single-file full reference

Maintained as the canonical index for external context. Update this file when adding major docs or changing deploy topology.

Included:

docs/CAF_CURRENT_STATE_CONTEXT_PACK.md

CAF Current State Context Pack

Generated: 2026-07-10

Authority: This document is derived from the live repository (`src/` , `migrations/` , `apps/review/` , `services/`). When it conflicts with older docs, **source wins**.

Purpose: Operational, file-path-specific map of CAF as it exists today — for product owners, engineers, operators, and external LLMs (ChatGPT, Claude, Cursor).

PDF volumes: For ChatGPT upload limits, use the four split files in `docs/volumes/` (regenerate PDFs via `npm run export:doc-pdfs`).

1. Executive summary

What CAF is

CAF (Content Automation Framework) is a **content automation / content operations platform** that turns research inputs, evidence, signal packs, creative intelligence, and planning decisions into **generated, QC-checked, rendered, reviewed, publishable, and learnable** social content.

The production loop:

```
Inputs / evidence → signal pack → planned jobs → decision engine →  
content jobs  
→ LLM drafts → QC / risk → diagnostics → rendering → human review  
→ rework → publishing → performance metrics → learning rules
```

Problem it solves

Operators run multi-platform content brands (e.g. SNS, Cuisina) at scale: ingest competitor/top-performer research, plan ideas per flow type, generate copy and media, enforce brand/risk gates, route through human review, publish to Meta, and close the loop with performance-driven learning.

Who uses it

Persona	Surface
Marketer / brand owner	Review app marketer funnel (/workspace, /brand/[slug]/*)
Content operator / reviewer	Review workbench (/review, /t/[task_id])
Pipeline engineer	Core API, CLI (npm run process-run), admin HTML
Inputs / research operator	Admin inputs pages, /v1/inputs-* APIs
External workers	n8n publish webhooks, renderer/video-assembly sidecars

Source of truth

- **PostgreSQL schema** `caf_core` — especially `content_jobs.generation_payload`
- **Core API** (`src/server.ts` , Fastify) — not the Review app
- **Migrations** (`migrations/*.sql`) — schema truth over prose docs

Mature vs experimental (honest)

Mature / production-used	Partial / gated / experimental
Run planning + job pipeline	Product image flows (FLOW_IMG_*) — blocked at generation
Standard carousel (FLOW_CAROUSEL)	Full composite saliency text-placement automation (designed, not built)
QC + risk policies + human review	Project risk_rules — config only, not QC-enforced
HeyGen video flows	Scene assembly (Sora) — wired but provider-dependent
Inputs → signal pack funnel	Stage-3 structured idea picker at plan time (partial)
Publications + Meta executor	CAF_PUBLISH_EXECUTOR=none default (n8n external)

Learning rules (planning + generation)	Global learning rules (disabled in compiler)
Review app workbench	Marketer funnel — growing, not all pipeline controls
Manual mimic carousel	Why Mimic + New Visual — newer, actively evolving
Brand Visual System (BVS) + brand bibles	BVS invented plates — <code>template_bg</code> + BVS only

2. Platform components

Component	Path	Purpose	State owned	Depends on	Key env
CAF Core API	<code>src/</code>	Orchestration, HTTP, business logic	<code>caf_core.*</code> tables	Postgres, OpenAI, providers	DATABAS
Postgres <code>caf_core</code>	<code>migrations/</code>	All persistent state	Everything	—	DATABAS
Review app	<code>apps/review/</code>	Operator + marketer UI (client)	None (proxies Core)	Core API, renderer	CAF_COR RENDERE
Admin HTML	<code>src/routes/admin.ts</code>	Legacy ops workbench	None	Core	Same as:
Renderer	<code>services/renderer/</code>	Puppeteer + HBS → PNG slides	Temp files	Templates from Core or disk	PORT, RE
Video assembly	<code>services/video-assembly/</code>	ffmpeg stitch/mux/burn	Temp uploads	Supabase	SUPABAS
Media gateway	<code>services/media-gateway/</code>	Single port proxy to renderer + video	None	Child processes	PORT, SP
Storage	Supabase	Asset URLs (carousel, video, mimic plates)	Bucket objects	—	SUPABAS SUPABAS
OpenAI	external	LLM generation, TTS, Sora, vision	None	API key	OPENAI_
HeyGen	external	Avatar / video agent renders	Provider session IDs	API key	HEYGEN_

			in render_state		
BFL / DashScope / NVIDIA	external	Mimic image edit / T2I	None	Provider keys	MIMIC_I BFL_API
Meta Graph	external	IG/FB publish	Placement rows	Page tokens	CAF_PUB CAF_MET
Apify / scrapers	external	Evidence ingest	Scraper run rows	APIFY_API_TOKEN	Partial
Document AI	external	Carousel OCR for mimic overlay seed	None	GCP creds	DOCUMEN
n8n / external publish workers	external	Legacy publish execution	Callback to Core	Webhook	CAF_PUB

Deployment: Production Review is embedded at <https://caf-core.fly.dev/admin/workbench> (not a separate Vercel deploy). Ship Review changes with `fly deploy -a caf-core` from repo root.

3. End-to-end funnel

Stage map

Stage	Trigger	Main files	DB tables	Input
Evidence upload	POST /v1/inputs-evidence/upload	inputs-evidence.ts, repos	inputs_evidence_imports, inputs_evidence_rows	XLS, nor
Scraper runs	POST /v1/inputs-sources/...	inputs-scrapers.ts	inputs_scraper_runs, inputs_source_rows	Apit evic
Insights passes	POST /v1/inputs-processing/.../run-*	inputs-processing.ts	inputs_evidence_row_insights	Row insig
Signal pack build	POST .../build-signal-pack	processing + signal-packs	signal_packs, ideas	Imp → p
Run create	POST /v1/runs	runs.ts, run-orchestrator.ts	runs	proj sigr

Materialize jobs	POST .../jobs Or .../candidates	runs.ts, run-candidates-materialize.ts	runs.planned_jobs_json	pac row
Start run	POST .../start	run-orchestrator.ts	content_jobs, runs	plar
LLM generation	pipeline auto / POST .../generate	llm-generator.ts, job-pipeline.ts	job_drafts, generation_payload	pro gene
Mimic prep	pipeline	mimic-draft-prep.ts, new-visual-carousel-prep.ts	generation_payload.mimic_v1	refe moc
QC	pipeline auto	qc-runtime.ts	generation_payload.qc_result	out che
Render	pipeline auto	job-pipeline.ts, render services	assets, render_state	pay mec
Human review	operator	Review app → Core /decide	editorial_reviews	dec ove
Rework	NEEDS_EDIT	rework-orchestrator.ts	new draft attempt	fee reg
Publish	POST .../publications/.../start	publications.ts, meta-graph-publish.ts	publication_placements	app plat
Learning	cron / manual APIs	learning.ts, editorial-learning.ts	learning_rules, learning_observations	out rule

Run lifecycle (exact strings)

CREATED → PLANNING → PLANNED → GENERATING → (RENDERING) → (REVIEWING) → COMPLETED | FAILED | CANCELLED

Set in: migrations/002_project_config_and_runs.sql ,
src/repositories/runs.ts , run-orchestrator.ts , job-pipeline.ts ,
runs.ts .

Job lifecycle (exact strings — no DB CHECK on content_jobs.status)

PLANNED → GENERATING → GENERATED → (RENDERING) → IN_REVIEW |
BLOCKED | REJECTED | NEEDS_EDIT | APPROVED | FAILED

Also observed: READY_FOR_REVIEW (legacy progress counting), QC_FAILED (only
pipeline.ts /full endpoint, not main pipeline).

4. Domain model and IDs

Entities

Entity	Description	Primary key pattern
Project	Named content brand	project_id (uuid), project_slug (text, e.g. SNS)
Run	One execution cycle	run_id = {PROJECT}_{period} e.g. SNS_2026W09
Signal pack	Research bundle for a run	signal_pack_id (uuid/text per schema)
Candidate / planned row	Idea × flow in memory or planned_jobs_json	candidate_id = {run_id}_{platform}_{NNNN} or {base}_{flow_type}
Content job	Atomic executable unit	task_id = {run_id}_{platform}_{flow_type}_row{NNNN}_{variation}
Job draft	One LLM attempt	draft_id = d_{random12}
Asset	Rendered media artifact	asset_id = {candidate_id}_{ASSET_TYPE}_v{version}
Editorial review	Human decision	(project_id, task_id, review_id)
Publication placement	Scheduled post	uuid row in publication_placements
Learning rule	Structured behavior change	uuid in learning_rules
Evidence import	XLSX/scrapper batch	inputs_evidence_imports.id
Evidence row	Single post/account row	per import
Evidence insight	Tier analysis on a row	inputs_evidence_row_insights

Creative intelligence asset	Archived top-performer media	creative_intelligence_assets
Brand bible	BVS source per project	brand_bibles versioned rows
Brand profile	Marketer voice/strategy	brand_profiles

Join pattern

All job-related tables join on **(project_id, task_id)** — text IDs, not UUID FKs to `content_jobs`.

Legacy / dual-write

Legacy	Canonical	Helper
<code>runs.candidates_json</code>	<code>runs.planned_jobs_json</code>	<code>readRunPlannedJobsJson()</code> in <code>jobs-json-compat.ts</code>
<code>signal_packs.ideas_json</code>	structured idea tables	both may coexist
Old flow type names	<code>canonical-flow-types.ts</code>	<code>LEGACY_FLOW_TYPE_TO_CANONICAL</code>

5. Database schema map

Grouped by domain (79 migrations through `078_brand_bibles.sql`):

Core pipeline

`projects`, `runs`, `content_jobs`, `job_drafts`, `assets`,
`job_state_transitions`, `signal_packs`, `signal_pack_ideas`,
`signal_pack_selected_ideas`, `ideas`, `idea_grounding_insights`

Writers: `run-orchestrator.ts`, `job-pipeline.ts`, `runs.ts`

Readers: all routes keyed by `task_id`

Review / QC / validation

`editorial_reviews`, `diagnostic_audits`, `auto_validation_results`,
`qc_checklists`, `qc_flow_profiles`

Writers: `validation-router.ts`, review routes, `qc-runtime.ts` (payload only)

Flow engine / prompts

flow_definitions , prompts , prompt_versions , prompt_schemas ,
carousel_templates , risk_policies

Writers: migrations seed, flow-engine.ts admin

Readers: llm-generator.ts , qc-runtime.ts

Project configuration

project_strategy , brand_constraints , product_profiles ,
platform_constraints , project_system_constraints ,
project_brand_assets , brand_profiles , brand_bibles ,
heygen_config , project_integrations , risk_rules (not QC-enforced)

Inputs / evidence

inputs_evidence_imports , inputs_evidence_rows ,
inputs_evidence_row_insights , inputs_evidence_packs ,
inputs_processing_profiles , inputs_source_rows ,
inputs_scraper_config , inputs_scraper_runs , inputs_idea_lists ,
inputs_ideas , evidence_media_assets , insights_packs

Creative intelligence / mimic

creative_intelligence_assets , creative_intelligence_analyses ,
creative_intelligence_insights (migration 055+)

Learning / performance

learning_rules , suppression_rules , learning_observations ,
learning_hypotheses , learning_insights ,
learning_generation_attribution , performance_metrics ,
performance_ingestion_batches , run_content_outcomes ,
job_outcomes (075), llm_approval_reviews

Publishing

publication_placements , project_integrations

Operations / audit

api_call_audit , run_context_snapshots , run_output_reviews

6. Lifecycle map

Runs

See §3. Terminal: `COMPLETED` , `FAILED` , `CANCELLED` .

Content jobs

See §3. Editorial terminal: `APPROVED` , `REJECTED` . Rework re-enters `GENERATING` .

Editorial decisions (`editorial_reviews.decision`)

`APPROVED` | `NEEDS_EDIT` | `REJECTED`

Publication placements

`draft` → `scheduled` → `publishing` → `published` | `failed` | `cancelled`

Learning rules

`pending` → `active` | `superseded` | `rejected` | `expired`

Provider render state (`generation_payload.render_state`)

HeyGen: `video_id` , `session_id` — retry guard via

`hasActiveProviderSession()` (`content-job-render-state.ts`).

Scene assembly: `scene_bundle` progress, Sora job IDs.

Carousel: `isCarouselRenderComplete()` .

QC recommended routes (`qc_result.recommended_route`)

`AUTO_PUBLISH` | `BLOCKED` | `DISCARD` | `REWORK_REQUIRED` | `HUMAN_REVIEW`

Default `CAF_REQUIRE_HUMAN_REVIEW_AFTER_QC=true` remaps clean QC to `HUMAN_REVIEW` .

7. Critical JSON contracts

`content_jobs.generation_payload` (main integration contract)

Slice	Meaning	Read/write helpers
-------	---------	--------------------

Plan-time base	schema_version, signal_pack_id, candidate_data, prompt binding	buildPlannedGenerationPayloadBase() (stage- contract.ts)
generated_output	LLM output body	pickGeneratedOutput(), hasGeneratedOutput()
qc_result	QC pass/fail, findings, route	pickStoredQcResult(), mergeGenerationPayloadQc() (only writer)
render_state	Provider sessions, carousel progress	pickRenderState(), hasActiveProviderSession(), isMidProviderPhase()
mimic_v1	Mimic render truth	pickMimicPayload(), MIMIC_PAYLOAD_KEY
mimic_carousel_package	Draft package for TP- grounded carousels	pickMimicCarouselDraftPackage() — all isTpGroundedCarouselRenderFlow()
carousel_package	Standard FLOW_CAROUSEL only	draft-package-contract.ts
bvs_v1	Frozen brand bible snapshot	parseBvsV1(), attachBvsToPlannedPayload()
mimic_job_grounding	Plan-time mimic grounding	mimic-job-grounding.ts
semantic_contract_v1	Cross-flow semantic constraints	semantic-contract.ts
content_display	Display metadata for Review	content-display-metadata.ts
layout_qc	Post-render layout QA blob	mimic-composite-layout-qa.ts
docai_layer_positions	Inside mimic_v1 — reviewer text boxes	mimic-docai-layer-positions.ts
scene_bundle	Scene assembly progress	scene-pipeline types
schema_validation_warnings	Non-fatal LLM schema issues	when CAF_OUTPUT_SCHEMA_VALIDATION_MODE=warn
Publish snapshots	URLs after approval	validation-router.ts

mimic_v1 key fields

```
// src/domain/mimic-payload.ts (abridged)
{
  schema_version: 1,
  execution_mode?: "classic" | "why_mimic" | "new_visual",
  mode: "image_full" | "template_bg" | "carousel_visual",
  reference_items[], visual_guideline, slide_plans[],
  flux_image_prompts?, docai_layer_positions?,
  bvs_enabled?, bvs_bible_snapshot?, bvs_render_plan?,
  slide_intelligence?, brand_execution_brief?
}
```

bvs_v1 key fields

```
// src/domain/bvs-v1.ts
{ schema_version: "bvs_v1", enabled: boolean, bible_version,
  bible_snapshot }
```

Signal pack JSON (high-signal fields)

- overall_candidates_json — raw idea rows
- visual_guidelines_pack_v1 — merged styling for mimic
- mimic_mode_overrides — per-insight reviewer mode overrides
- derived_globals_json — CI styling merge output

Risky untyped areas

- Ad-hoc reads of candidate_data nested fields in Review components
- Legacy ideas_json on older signal packs
- render_manifest (commented in migrations, sparse usage)
- Some admin HTML forms bypass typed helpers

8. Flow types and content formats

Canonical text/copy flows (canonical-flow-types.ts)

```
FLOW_CAROUSEL , FLOW_ANGLE , FLOW_STRUCTURE , FLOW_CTA , FLOW_HOOKS ,
FLOW_TEXT
```

Video flows

Flow	Status	Generate	Render	Review
FLOW_VID_SCRIPT	Production	OpenAI script	HeyGen	Video edits panel
FLOW_VID_PROMPT	Production	OpenAI prompt	HeyGen Video Agent	HeyGenReviewEdits
FLOW_VID_PROMPT_NO_AVATAR	Production	OpenAI	HeyGen no-avatar	HeyGenReviewEdits
FLOW_VID_SCENES / FLOW_SCENE_ASSEMBLY	Provider-dependent	Scene scripts	Sora clips + ffmpeg concat	Video review
FLOW_PRODUCT_* (6 types)	Production	Product prompts	HeyGen	Video review

Carousel flows

Flow	Status	Notes
FLOW_CAROUSEL	Production	carousel_package, HBS templates, CarouselBrandStylingPanel
FLOW_TOP_PERFORMER_MIMIC_CAROUSEL	Production (gated render)	mimic_v1, reference frames, compare UI in Review
FLOW_VISUAL_FIRST_CAROUSEL	Production (evolving)	execution_mode: "new_visual" — no TP references, BVS-driven
FLOW_WHY_MIMIC_CAROUSEL	Production (newer)	SIL-driven, execution_mode: "why_mimic"
FLOW_TOP_PERFORMER_MIMIC_IMAGE	Gated	image_full mode, single frame
FLOW_TOP_PERFORMER_MIMIC_VIDEO	Alias	Routes to HeyGen video planning

Product image flows (FLOW_IMG_*)

Registered in flow engine; **blocked at LLM** with
PRODUCT_IMAGE_FLOW_NOT_READY_MESSAGE .

Offline / excluded (offline-flow-types.ts)

Reel_Script , FLOW_REEL_SCRIPT , hook variation flows — excluded from planning.

Per-flow requirements

Flow family	Env / config
-------------	--------------

Mimic carousel	MIMIC_IMAGE_ENABLED=1, provider API key, archived reference media (except new_visual)
New visual	BVS recommended; analysis_t2i input mode
HeyGen	HEYGEN_API_KEY, project heygen_config
Scene assembly	OPENAI_API_KEY, SCENE_ASSEMBLY_CLIP_PROVIDER, VIDEO_ASSEMBLY_BASE_URL
Standard carousel	RENDERER_BASE_URL, templates

9. Review app and operator workflow

Architecture

- **Next.js** in `apps/review/` — **client only**
- All data via `caf-core-client.ts` → `${CAF_CORE_URL}/v1/...`
- 78 proxy routes under `apps/review/src/app/api/`
- Embedded in Core Fly image at `/admin/workbench`

Two audiences

1. **Marketer:** `/workspace` → `/brand/[slug]/*` (profile, research, ideas, content, publishing, performance)
2. **Operator:** `/review`, `/runs`, `/pipeline`, `/publish`, `/learning`
(enable via `?debug=1` or operator sidebar)

Queue and job detail

- `GET /api/tasks` → Core review queue
- `GET /api/task/[task_id]` → full job + assets + payload slices
- `TaskReviewClient.tsx` orchestrates viewers and edit panels

Carousel editing (`FLOW_CAROUSEL`)

- `CarouselSlider.tsx` — live preview via `/api/renderer/preview-live-slide`
- `CarouselEdits.tsx` + `CarouselBrandStylingPanel.tsx` — typography, palette, logo, frame
- Rework overrides sent on decision

Mimic carousel editing (TP-grounded flows)

- `MimicCarouselLayerEditorPanel.tsx` — 3-column: copy | DocAI layer editor | typography/BVS
- **Save layout** → `mimic-docai-layer-positions`

- **Reprint text** (cheap) → `reprint-text-overlay` — Puppeteer on stored plates
- **Regenerate slides** (expensive) → Flux/Qwen when picture is wrong
- `FLOW_TOP_PERFORMER_MIMIC_CAROUSEL` : original vs generated compare
- `FLOW_VISUAL_FIRST_CAROUSEL` : same editor, **no** compare row

Decisions (`DecisionPanel.tsx`)

`APPROVED | NEEDS_EDIT | REJECTED` → Core `/v1/review-queue/:slug/decide`

Auto-upgrades `APPROVED` → `NEEDS_EDIT` if unsaved edits exist.

Rework options

`rewrite_copy` , `regenerate` , `carousel_rework_change_template` , `slide_rework_indices` , skip video/image regen flags.

Publishing UI

`/publish` — schedule placements, n8n payload export, Meta start (when configured).

Learning UI

`/learning/*` — rule inbox, apply/retire, context preview, analyzers, observatory.

Review owns vs Core owns

Review owns	Core owns
UX, live preview assembly	<code>content_jobs.status</code> , <code>generation_payload</code>
Proxy auth headers	QC, render orchestration
Background job toasts (polling)	<code>editorial_reviews</code> , <code>assets rows</code>
Marketer funnel navigation	Publication execution

Contract guard

`src/routes/review-contract.test.ts` — every `REVIEW_CRITICAL_PATHS` must exist in Core routes (CI breakage detector).

Env

`CAF_CORE_URL` , `CAF_CORE_TOKEN` , `PROJECT_SLUG` , `REVIEW_WRITE_TOKEN` , `RENDERER_BASE_URL` , `NEXT_PUBLIC_APP_URL`

10. Inputs pipeline and signal pack creation

Implemented

1. XLSX upload → `inputs_evidence_imports` + rows
2. Scraper config/runs (Apify) → same evidence shape
3. Evidence packs (multi-platform)
4. Processing profile (criteria, models, caps)
5. Insight tiers: `broad_llm` , `top_performer_deep` , `top_performer_video` , `top_performer_carousel`
6. Document AI OCR on carousel tiers
7. Rating/synthesis → `overall_candidates_json`
8. Idea lists (`inputs_idea_lists` , `inputs_ideas`)
9. Signal pack build (from import or idea list)
10. RTP summary, QC flow profiles, API audit

Partial / TBD (from roadmap + code)

- Structured Stage-3 idea picker at plan time (idea lists exist; full planner UX incomplete)
- Rich explainable idea scoring with brand/risk eligibility gates
- Persist `decideGenerationPlan` I/O snapshot per run
- Optional persisted candidate audit rows
- HTML/platform summary folding
- Review app: upload + inspect only (no processing controls)

Signal pack → run

`POST /v1/runs` with `signal_pack_id` → materialize `planned_jobs_json` → `startRun()` → `content_jobs` at `PLANNED` .

11. Creative intelligence and mimic system

Top-performer ingest

`src/routes/creative-intelligence.ts` — ingest assets, run vision analysis, store insights, styling packs, mimic-carousel-template mint/apply.

Tables: `creative_intelligence_assets` , analyses, insights (migration 055+).

Visual guidelines on signal packs

`visual_guidelines_pack_v1` merged into generation creation pack;
`derived_globals_json` from styling merge.

Mimic reference resolution (`mimic-draft-prep.ts`)

- Archives frames from CI assets
- Filters promo/video slides
- Builds `reference_items` , `slide_plans` , `visual_guideline`
- **New visual bypasses this** (`new-visual-carousel-prep.ts`)

Mode classification (`mimic-mode-classifier.ts`)

Priority: reviewer override → Nemotron `mimic_evaluation` → heuristics (`mimic-text-heavy.ts`)

Mode	Visual strategy	Text strategy
<code>template_bg</code>	Background plate (reference strip or BVS-invented T2I)	HBS/DocAI overlay
<code>carousel_visual</code>	Per-slide art-only plate (<code>full_bleed</code>)	HBS/DocAI overlay
<code>image_full</code>	Single image-edit pass	Copy on-image from LLM

Execution modes on `mimic_v1`

<code>execution_mode</code>	Flow	Behavior
<code>classic</code>	<code>FLOW_TOP_PERFORMER_MIMIC_CAROUSEL</code>	TP fidelity + references
<code>why_mimic</code>	<code>FLOW_WHY_MIMIC_CAROUSEL</code>	SIL-driven strategic copy/prompts
<code>new_visual</code>	<code>FLOW_VISUAL_FIRST_CAROUSEL</code>	Idea+BVS, empty references, always <code>carousel_visual</code>

Brand Visual System (BVS)

Brand bible (`brand_bibles` table, `brand_bible_v1` schema):

- Visual mode, palette, motifs, asset roles (`slide_frame` , `logo` , `style_reference` , ...)
- `application_guide.mimic_policy` / `original_policy`
- API: brand bible CRUD on review-queue routes

Job snapshot (`generation_payload.bvs_v1`):

- Stamped at plan when `candidate_data.use_brand_visual_system === true`
- Visual-first lane defaults BVS on
- `bvs_render_plan` on `mimic_v1` for `template_bg` invented plates

Render overlays (`bvs-render-overlays.ts`):

- Logo/frame auto-overlay for `template_bg` + `bvs_enabled`

- Palette injection into carousel theme colors

Text overlay / DocAI / reprint

Invariant: TP-grounded carousels do **not** bake LLM copy into Flux

(`mimicCarouselTextViaFlux` hardcoded false in `job-pipeline.ts`).

Pipeline:

```
Art-only plate → DocAI seed from reference OCR → Puppeteer HBS
→ reviewer docai_layer_positions → reprint-text-overlay →
CAROUSEL_IMAGE assets
```

Templates: `carousel_mimic_bg.hbs` , `MIMIC_FULL_BLEED_RENDER_TEMPLATE` .

Layout QA loop (`mimic-post-render-layout-loop.ts`): optional automated reposition + reprint (`MIMIC_LAYOUT_QA_*`).

Composite saliency QA (full automation): documented in

`MIMIC_TEXT_PLACEMENT_AUTOMATION.md` , **not implemented**.

Automated vs manual

Automated	Manual
CI ingest + vision	TP selection for ingest
Mode classification	<code>mimic_mode_overrides</code> on signal pack
Reference resolve + plate generation	Brand bible editing
DocAI box seed	Layer editor geometry
Optional plate text QA + layout QA reprint	Editorial approve/reject
BVS snapshot at plan	Per-slide Flux regen when picture wrong

Current limitations

- `MIMIC_IMAGE_ENABLED=false` by default — render blocked without explicit enable
- New visual docs still describe old TP-grounded model in places
- Duplicate migration prefix `071_*` (slide intelligence + new visual prompt)
- Why Mimic requires substantive SIL (`WHY_MIMIC_REQUIRE_SUBSTANTIVE_SIL`)

12. QC, risk, validation, and quality gates

QC runtime (`qc-runtime.ts` → `runQcForJob`)

1. **Checklist** from `flow_definitions.qc_checklist_name` → `qc_checklists`
2. **Copy quality** patterns (hashtags, tone, etc.)
3. **risk_policies** — global + flow-scoped via `listRiskPoliciesForJob()`
4. **brand_constraints.banned_words** — same keyword scan as policies

NOT enforced by QC

- **risk_rules** (project CSV rows) — zero references in `qc-runtime.ts`
- Honesty endpoint: `GET /v1/projects/:slug/risk-qc-status`

Pass/fail

`qc_passed` = no blocking checklist failures AND no CRITICAL risk findings.

Routing (`validation-router.ts`)

Maps `recommended_route` to job status; `finalJobStatusAfterRender()` → `IN_REVIEW` .

Other gates

- **Output schema validation** — `CAF_OUTPUT_SCHEMA_VALIDATION_MODE` (`off` | `warn` | `strict`)
- **Draft package contract** — `CAF_DRAFT_PACKAGE_CONTRACT_MODE`
- **Diagnostic audits** — machine quality evaluation
- **Auto-validation** — `auto_validation_results` table
- **Post-approval LLM review** — `llm_approval_reviews` , upstream recommendations via `insertLlmApprovalReview()`
- **Human review gate** — `CAF_REQUIRE_HUMAN_REVIEW_AFTER_QC=true` default

13. Learning loop

Facade (mandatory entry point)

`src/services/learning-rule-selection.ts`

Function	When	Rules
<code>getLearningRulesForPlanning()</code>	Decision engine scoring	<code>BOOST_RANK</code> , <code>SCORE_BOOST</code> , <code>SCORE_PENALTY</code> only

<code>getLearningContextForGeneration()</code>	LLM prompt build	generation family / GUIDANCE action types
--	------------------	---

Planning path

`decision_engine/index.ts` → `applyLearningBoosts()` multiplies `pre_gen_score` .

Sources: `editorial-learning.ts` , `performance-learning.ts` , `market-learning.ts` .

Generation path

`learning-context-compiler.ts` → `merged_guidance` injected in `llm-generator.ts` .

Preview: `GET /v1/learning/:slug/context-preview` .

Global rules disabled in compiler.

Evidence → rules loops

Loop	Trigger	Output
Editorial (B)	<code>POST .../editorial-analysis</code>	Pending <code>SCORE_PENALTY</code> rules
Performance (C)	<code>performance ingest/CSV/analysis</code>	Ranking rules from metrics
LLM approval	<code>POST .../llm-review-approved</code>	Upstream recommendations + hints
Manual	<code>apply/retire/dismiss APIs</code>	<code>pending</code> → <code>active</code>

Attribution / snapshots

- `learning_generation_attribution` — which rules affected a generation
- `run_context_snapshots` via `setRunContextSnapshot()` — prompt versions + fingerprints (failures logged, never abort run)
- `job_outcomes` (075) — `publish` → performance anchor

Tables

`learning_rules` , `learning_observations` , `learning_hypotheses` , `learning_insights` , `performance_metrics` , `llm_approval_reviews`

14. Publishing and integrations

Model

`publication_placements` — one row per scheduled post: caption/media snapshots, platform, format (`carousel` | `video` | `unknown`).

Lifecycle

`draft` → `scheduled` → `publishing` → `published` | `failed` | `cancelled`

Executor modes (`CAF_PUBLISH_EXECUTOR`)

Mode	Behavior
<code>none</code> (default)	Start claims placement; returns n8n payload; external worker calls <code>/complete</code>
<code>dry_run</code>	Fake IDs for plumbing tests
<code>meta</code>	Core calls <code>publishPlacementToMeta()</code> directly

Meta integration (`meta-graph-publish.ts`)

Facebook feed/photos/video; Instagram single/carousel/reels; polls container status; re-signs Supabase URLs for Meta fetch.

Credentials: `project_integrations` (`META_FB` , `META_IG`) + env overrides `CAF_META_*` .

Account aliasing: `CAF_META_ACCOUNT_SOURCE_MAP` (e.g. `CUISINA=SNS`).

n8n payload

`publication-n8n-payload.ts` — legacy field names matching `Publish_Carousel_IG_FB` / `Publish_Video_IG_FB` .

On success

`appendPublicationResultToJob()` + `upsertJobOutcomeOnPublish()` → closes loop to `job_outcomes` / performance.

Project integrations

`src/routes/project-integrations.ts` — CRUD + test for external credentials.

15. API surface map

Registration hub: `src/server.ts`

Domain	Route file	Key endpoints	Auth
--------	------------	---------------	------

Health	v1.ts	/, /health, /readyz, /health/rendering	Optional bearer
Decisions / jobs	v1.ts	/v1/decisions/plan, /v1/jobs/ingest, GET .../jobs/:slug/:task_id	Token if required
Review queue	v1.ts	/v1/review-queue/... (large surface: decide, regen, reprint, docai, mimic-mode)	Token
Runs	runs.ts	CRUD, /jobs, /start, /process, /render, /cancel	Token
Pipeline	pipeline.ts	/generate, /qc, /diagnose, /full, /rework	Token
Projects	project-config.ts	strategy, brand, risk-qc-status, flow-types, brand-assets, heygen	Token
Flow engine	flow-engine.ts	flows, prompts, schemas, templates, qc-checks, risk-policies	Token
Signal packs	signal-packs.ts	upload, ideas, select-ideas, mimic-mode overrides	Token
Inputs evidence	inputs-evidence.ts	upload, list, rows	Token
Inputs scrapers	inputs-scrapers.ts, inputs-sources.ts	workbook, scraper runs	Token
Inputs processing	inputs-processing.ts	profile, insight passes, build-signal-pack, idea-lists	Token
Evidence read	evidence-insights-read.ts	/v1/evidence, /v1/insights	Token
Creative intel	creative-intelligence.ts	ingest, analyses, styling, mimic-template	Token
Learning	learning.ts	rules, observations, performance, editorial-analysis, llm-review	Token
Publications	publications.ts	CRUD, start, complete, n8n-payload	Token
Integrations	project-integrations.ts	Meta etc.	Token
Admin	admin.ts, mimic-text-overlay-lab-routes.ts	HTML workbench + lab APIs	Token
Renderer templates	renderer-templates.ts	/api/templates	Often open
Review proxy	review-proxy.ts	Non-Core paths → embedded Next.js	Review exempt

Auth: When `CAF_CORE_REQUIRE_AUTH` + `CAF_CORE_API_TOKEN` , Bearer or `x-caf-core-token` on Core paths.

16. Environment and deployment

Local minimum

```
DATABASE_URL=postgres://...
OPENAI_API_KEY=sk-...
# Optional for full loop:
RENDERER_BASE_URL=http://localhost:3333
CAF_CORE_URL=http://localhost:3847 # Review app
```

Production groups

See `ENV_AND_SECRETS_INVENTORY.md` and `src/config.ts` (Zod-validated).

Group	Notable vars
Core	DATABASE_URL, CAF_RUN_MIGRATIONS_ON_START, CAF_CORE_API_TOKEN, CAF_REVIEW_ENABLED, CAF_PUBLIC_URL
Planning	DECISION_ENGINE_VERSION, DEFAULT_MAX_CAROUSEL_JOBS_PER_RUN, score weights
QC	CAF_REQUIRE_HUMAN_REVIEW_AFTER_QC, CAF_OUTPUT_SCHEMA_VALIDATION_MODE
Rendering	RENDERER_BASE_URL, VIDEO_ASSEMBLY_BASE_URL, concurrency/timeouts
Mimic	MIMIC_IMAGE_ENABLED, MIMIC_IMAGE_PROVIDER, BFL_API_KEY, MIMIC_LAYOUT_QA_*
Video	HEYGEN_API_KEY, SCENE_ASSEMBLY_CLIP_PROVIDER, SORA_*
Publish	CAF_PUBLISH_EXECUTOR, CAF_META_*, META_GRAPH_API_VERSION
Storage	SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_ASSETS_BUCKET
Inputs	APIFY_API_TOKEN, DOCUMENT_AI_*, CREATIVE_INTEL_*

Deploy

- **Core + Review:** `fly deploy -a caf-core` (Dockerfile builds Next.js standalone)
 - **Renderer / media:** separate Fly apps (`fly.caf-renderer.toml` , `fly.toml`)
 - **Migrations:** `npm run migrate` or on startup when enabled
-

17. Current maturity matrix

Feature / subsystem	Maturity	Evidence	Known gaps	Main files
Standard carousel	Production	Full pipeline + Review editor	Template variety	carousel-render-pack.ts, CarouselEdits.tsx
Carousel editor	Production	Live preview, brand styling panel	—	CarouselSlider.tsx, preview-slide/route.ts
Mimic carousel	Production (gated)	Full prep + render + layer editor	Default MIMIC_IMAGE_ENABLED=false	mimic-draft-prep.ts, MimicCarouselLayerEditorP
New visual carousel	Active development	new-visual-carousel-*.ts, migration 070/071	Docs lag	new-visual-carousel-prep.
Why Mimic carousel	Newer production	migration 074, SIL	Requires substantive SIL	why-mimic-carousel-flow-t
BVS / brand bible	New (migration 078)	brand-bibles.ts, bvs-v1.ts	Docs not updated	brand-bible.ts, bvs-render-plan.ts
Mimic image (single)	Gated	mimic-image-job.ts	Same env gate	mimic-image-provider.ts
Mimic text placement automation	Partial	Layout QA loop only	Composite saliency not built	mimic-post-render-layout-
HeyGen video	Production	heygen-renderer.ts	Provider quotas	HEYGEN_API_KEY
Scene assembly	Provider-dependent	scene-pipeline.ts	Sora availability	sora-scene-clips.ts
Inputs pipeline	Largely built	migrations 027–065+	Stage-3 picker partial	inputs-processing.ts
Creative intelligence	Production	migration 055	Downstream SIL docs thin	creative-intelligence.ts

Publishing	Production	Meta + n8n paths	Default executor none	publications.ts
Learning	Production	Facade + compiler	Global rules disabled	learning-rule-selection.t
QC / risk	Production	qc-runtime.ts	risk_rules not enforced	risk-qc-status.ts
Review app	Production	Embedded in Core	Marketer funnel incomplete vs admin	apps/review/
Admin config	Production	admin.ts, project-config routes	HTML legacy mixed with API	project-config.ts
Product image flows	Placeholder	Blocked at LLM	Not wired	product-flow-types.ts

18. Repo map for future AI agents

Goal	Start here	Do not casually change
Run planning / scoring	src/decision_engine/, run-orchestrator.ts	ID patterns, planned_jobs_json contract
Job execution	job-pipeline.ts	Lifecycle enums
LLM generation	llm-generator.ts, flow-engine repos	Prompt versions without migration
QC	qc-runtime.ts	Write qc_result except via mergeGenerationPayloadQc
Risk policies	flow-engine risk-policies, qc-runtime.ts	Assume risk_rules are enforced
Rendering	job-pipeline.ts, carousel-render-pack.ts, services/renderer/	HeyGen retry without hasActiveProviderSession
Review UI	apps/review/src/components/TaskReviewClient.tsx	Job state authority
Mimic	mimic-draft-prep.ts, mimic-carousel-render.ts, mimic-payload.ts	Conflate carousel_package with mimic_carousel_package
BVS	brand-bible.ts, bvs-v1.ts, bvs-render-plan.ts	—

New visual	new-visual-carousel-prep.ts, new-visual-carousel-execution.ts	Re-add TP references to visual-first
Inputs	inputs-processing.ts, inputs-evidence.ts	Signal pack JSON contracts
Publishing	publications.ts, meta-graph-publish.ts	Placement lifecycle strings
Learning	learning-rule-selection.ts facade only	Direct listActiveAppliedLearningRules imports
Env / config	src/config.ts	—
Review contract	review-contract.test.ts	Rename paths without updating test
Schema truth	migrations/	—

19. Non-negotiable invariants

1. **task_id** + **project_id** remain primary execution keys across **job_drafts** , **assets** , **editorial_reviews** , **job_state_transitions** , etc.
2. **Postgres caf_core** is source of truth; Review app is a client.
3. **content_jobs.generation_payload** is a versioned integration contract — coordinate pipeline, Review, admin consumers.
4. **QC qc_result** must be written only via **mergeGenerationPayloadQc()** (**generation-payload-qc.ts**).
5. **HeyGen/provider retries** must use **hasActiveProviderSession()** — no double-submit when **video_id** / **session_id** exist.
6. **Learning lookups** go through **learning-rule-selection.ts** facade — not direct compiler imports from new code.
7. **risk_policies** + **brand banned_words** are QC-enforced; **project risk_rules** are NOT (see **risk-qc-status**).
8. **Lifecycle status strings** must not be casually renamed (runs have SQL CHECK; jobs do not but consumers depend on strings).
9. **Migrations + source win** over docs when conflicting.
10. **mimic_carousel_package** is for TP-grounded carousel render flows only (**isTpGroundedCarouselRenderFlow()**); standard carousel uses **carousel_package** .
11. **Mimic LLM creation pack** must filter signal pack to the job's single planned idea (**mimicFlowOnly: true**).

- 12. **Mimic carousel text** is overlay-only — do not bake copy into Flux at render (enforced in `job-pipeline.ts`).
 - 13. **Upstream recommendations** must use `parseUpstreamRecommendations` + `insertLlmApprovalReview()` ; each item logged as `learning_observation` .
 - 14. **Run context snapshots** via `setRunContextSnapshot()` — failures logged, never abort run.
 - 15. **Review /v1/ contract** — changes to listed paths require `review-contract.test.ts` update in same PR.
-

20. Documentation drift and required updates

Still accurate

- `AGENTS.md` — invariants match code
- `docs/DOMAIN_MODEL.md` — ID patterns (minor column naming caveat)
- `docs/LIFECYCLE.md` — run states (job diagram omits `GENERATED`)
- `docs/QUALITY_CHECKS.md` , `docs/RISK_RULES.md` — `risk_rules` vs `risk_policies` asymmetry
- `docs/EXTERNAL_CONTEXT_PACK.md` — tier structure (needs new pack reference)
- `docs/CAF_INPUTS_PIPELINE_ROADMAP.md` — largely matches implementation
- `.cursor/rules/caf-domain-model.mdc` , `mimic-carousel-package.mdc` , `mimic-signal-pack-llm-filter.mdc`

Outdated or incomplete

Doc	Drift
MIMIC_FLOWS_COMPLETE_GUIDE.md	Updated July 2026 — still verify against current-state pack for edge cases
CREATIVE_INTELLIGENCE.md	Updated downstream lanes (SIL, new visual, Why Mimic, BVS)
MIMIC_IMAGE_FLOWS.md	Rewritten July 2026
CAF_CORE_COMPLETE_GUIDE.md	Updated July 2026 — points to current-state pack for latest truth
DATABASE_SCHEMA.md	Updated through migration 078
JOB_LIFECYCLE.md	Updated <code>QC_FAILED</code> / <code>READY_FOR_REVIEW</code> notes

Features in code, not documented

- Brand Visual System (`brand_bibles` , `bvs_v1` , `bvs_render_plan` , invented plates)
- `FLOW_WHY_MIMIC_CAROUSEL` + slide intelligence layer
- `FLOW_VISUAL_FIRST_CAROUSEL` as **new visual** (not TP replication)
- `execution_mode` on `mimic_v1`
(`classic` | `why_mimic` | `new_visual`)
- `job_outcomes` table (075)
- `CarouselBrandStylingPanel` + live preview overlays in Review
- Per-project mimic render settings (migrations 066–069)
- `MIMIC_CAROUSEL_TEXT_VIA_FLUX` ignored at render (hardcoded false)

Documented but not fully implemented

- `docs/MIMIC_TEXT_PLACEMENT_AUTOMATION.md` — composite saliency analyzer
- Project `risk_rules` enforcement (documented as config-only — correct)
- Stage-3 structured idea picker (roadmap TBD)
- Global learning rules in generation compiler

Suggested update priority (remaining)

1. `visual-first-carousel-flow-types.ts` header comment — align with `new_visual` semantics
2. `docs/layers/*.md` — spot-check when touching those layers (orchestration + generation updated July 2026)
3. Regenerate PDFs after doc edits: `npm run export:doc-pdfs`

Appendix A — ChatGPT upload bundle (2026-07)

For external LLM projects, upload **in order**:

Priority	File
1	<code>docs/CAF_CURRENT_STATE_CONTEXT_PACK.md</code> (this file) OR volumes 1–4
2	<code>AGENTS.md</code>
3	<code>docs/EXTERNAL_CONTEXT_PACK.md</code>
4	<code>apps/review/CHATGPT_CAROUSEL_REVIEW_CONSOLE_EDIT_GUIDE.md</code> (operator mimic workflow)
5	Topic add-ons: <code>docs/MIMIC_FLOWS_COMPLETE_GUIDE.md</code> (until updated), <code>docs/CREATIVE_INTELLIGENCE.md</code>

System prompt: Use the template in `docs/EXTERNAL_CONTEXT_PACK.md` § "System prompt template".

Regenerate PDFs: `npm run export:doc-pdfs` after doc changes.

Included: docs/REBUILD_FROM_DOCS.md

CAF Core — Rebuild from documentation

Purpose: Step-by-step guide for another team or repository to **stand up a working CAF Core stack** using only this repo's docs and code — without tribal knowledge.

Prerequisites: Node.js ≥ 20 , Docker (optional, for local Postgres), accounts for OpenAI and any media providers you enable.

Read first: [CAF CURRENT STATE CONTEXT PACK.md](#), [EXTERNAL CONTEXT PACK.md](#), [DOMAIN MODEL.md](#), [DATABASE SCHEMA.md](#).

What you are building

Component	Path	Port (default)	Required?
CAF Core API	repo root	3847	Yes
PostgreSQL	Docker or hosted	5432	Yes
Review app	apps/review	3000	Recommended — production embeds in Core Fly at /admin/workbench
Carousel renderer	services/renderer	3333	For carousel flows
Video assembly	services/video-assembly	3334	For video stitch/mux
Media gateway	services/media-gateway	3300 / 8080	Optional (combines renderer + assembly)

Core alone can run planning, generation, QC, and API-backed review **without** media workers — but carousel/video render steps will fail until workers are reachable.

Phase 1 — Database

Option A: Docker Compose (local)

```
docker compose up -d
```

Use the `DATABASE_URL` from `.env.example` comments (typically `postgresql://postgres:postgres@localhost:5432/caf_core`).

Option B: Hosted Postgres (Supabase, Fly Postgres, etc.)

Create an empty database. Set `DATABASE_URL` to the connection string (pooler URI mode works).

Apply schema

```
cp .env.example .env
# Edit DATABASE_URL
npm install
npm run migrate
```

Migrations create schema `caf_core` and all tables. Re-running migrate is safe.

Optional seed data

```
npm run seed:demo          # Demo project
npm run seed:canonical-flows
npm run seed:flow-engine    # Flow definitions + prompts
```

Phase 2 — Core API minimum config

In `.env` , set at minimum:

Variable	Required	Notes
<code>DATABASE_URL</code>	Yes	Postgres connection
<code>OPENAI_API_KEY</code>	Yes for LLM	Generation, QC helpers, mimic copy
<code>OPENAI_MODEL</code>	Recommended	Default chat model

For a **smoke test** without rendering:

```
npm run dev
curl http://localhost:3847/health
```

Recommended for full pipeline

Variable	Purpose
RENDERER_BASE_URL	Carousel worker (default http://localhost:3333)
VIDEO_ASSEMBLY_BASE_URL	ffmpeg worker (default http://localhost:3334)
SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_ASSETS_BUCKET	Asset uploads
HEYGEN_API_KEY	Avatar / video agent flows
MIMIC_IMAGE_ENABLED=1 + provider keys	Top-performer mimic render

Full list: `.env.example` , `ENV_AND_SECRETS_INVENTORY.md` , `src/config.ts`
(Zod schema = source of truth).

Phase 3 — Media workers

Carousel renderer

```
cd services/renderer
npm install
# Optional: CAF_TEMPLATE_API_URL=http://localhost:3847 (pull HBS from
Core)
npm start
# Listens on PORT (default 3333)
```

Core serves templates at `GET /api/templates/*` from `CAROUSEL_TEMPLATES_DIR`
(see `src/routes/renderer-templates.ts`).

Video assembly

```
cd services/video-assembly
npm install
# Set SUPABASE_* if uploading outputs
npm start
# Listens on PORT (default 3334)
```

Media gateway (production pattern)

```
cd services/media-gateway
npm install
node server.js
```

Child processes inherit env — set Supabase and mux vars on the gateway host. See `services/media-gateway/` and `fly.toml` variants.

Phase 4 — Review app

```
cd apps/review
npm install
```

Create `apps/review/.env.local` :

```
CAF_CORE_URL=http://localhost:3847
# If Core uses auth:
# CAF_CORE_TOKEN=<same as CAF_CORE_API_TOKEN>
# Optional:
# RENDERER_BASE_URL=http://localhost:3333
# PROJECT_SLUG=SNS
```

```
npm run dev
# http://localhost:3000
```

Review is a **client** — all job state comes from Core APIs.

Phase 5 — End-to-end workflow

Path A: Legacy XLSX intake

```
npm run start-run:xlsx -- path/to/signals.xlsx --project SNS
```

Or `POST /v1/signal-packs/upload` (multipart).

Path B: Inputs pipeline (evidence → signal pack)

1. Admin UI: `/admin/inputs` — upload evidence XLSX or run scrapers
2. `/admin/processing` — insights, profile, build signal pack
3. Create run linked to signal pack

See `docs/CAF_INPUTS_PIPELINE_ROADMAP.md`.

Run lifecycle (API)

1. Create run with `signal_pack_id` → status `CREATED`
2. `POST /v1/runs/:project_slug/:run_id/jobs` — materialize `planned_jobs_json`
3. `POST /v1/runs/:project_slug/:run_id/start` — plan + insert `content_jobs`
4. `POST /v1/runs/:project_slug/:run_id/process` — generate + QC (background)
5. `POST /v1/runs/:project_slug/:run_id/render` — render `GENERATED jobs`
6. Review via Review app or `/v1/reviews`
7. `POST /v1/publications/:project_slug/...` — publish placements

CLI alternative: `npm run process-run -- <run_id> --project SNS`

Phase 6 — Production deploy

Target	Config
Core	<code>fly.toml</code> , <code>Dockerfile</code> at repo root
Media gateway	<code>services/media-gateway/fly.toml</code>
Review	<code>apps/review/vercel.json</code> — Vercel

Checklist: `docs/FLY_PRODUCTION_CHECKLIST.md`

Production essentials:

- `CAF_CORE_REQUIRE_AUTH=1` + `CAF_CORE_API_TOKEN`
 - `CAF_PUBLIC_URL` — public Core URL for asset links
 - `RENDERER_BASE_URL` / `VIDEO_ASSEMBLY_BASE_URL` — stable worker URLs
 - Run `npm run migrate` on deploy or enable `CAF_RUN_MIGRATIONS_ON_START`
-

Phase 7 — Verify the stack

Check	Command / URL
Core health	<code>GET /health</code>
Rendering deps	<code>GET /health/rendering</code>
Review → Core	Review app <code>/api/health/core</code>
Renderer	<code>GET {RENDERER_BASE_URL}/health</code>
DB migrations	<code>SELECT * FROM caf_core.schema_migrations ORDER BY version</code>
Tests	<code>npm test (Vitest)</code>

What docs cannot replace

To **fully re-implement** behavior identical to this repo, you still need:

- **Source code** in `src/services/job-pipeline.ts` , `llm-generator.ts` ,
mimic modules, etc.
- **Flow engine seed data** — prompt templates and output schemas in DB
- **Migration history** — incremental schema evolution in `migrations/`
- **Integration tests** — `src/**/*.test.ts` , `src/routes/review-contract.test.ts`

For **architectural parity**, Tier 1 docs in [EXTERNAL CONTEXT PACK.md](#) are sufficient. For **behavioral parity**, treat this repository as the reference implementation.

Troubleshooting

Symptom	Likely cause
---------	--------------

start fails — no candidates	Run POST .../jobs first to materialize planned_jobs_json
Jobs stuck RENDERING	HeyGen poll / missing HEYGEN_API_KEY / RenderNotReadyError retry
Carousel render timeout	Renderer not running; check RENDERER_BASE_URL, CAROUSEL_RENDERER_SLIDE_TIMEOUT_MS
QC passes but risk expected	Project risk_rules not enforced — use risk_policies or brand bans
Review 401	CAF_CORE_TOKEN mismatch with Core CAF_CORE_API_TOKEN
Mimic disabled	MIMIC_IMAGE_ENABLED not set; flow type not in allowed list

See also

- [README.md](#) — quick start
- [API REFERENCE.md](#) — HTTP examples
- [TECH STACK.md](#) — stack detail
- [CAF CORE COMPLETE GUIDE.md](#) — merged reference

Included: docs/DOMAIN_MODEL.md

CAF Core — Domain model

Purpose: External-facing copy of the CAF domain rules (IDs, entities, lifecycles, joins). Cursor agents also use `.cursor/rules/caf-domain-model.mdc` ; **this file is for humans, ChatGPT, and other repositories.**

Pipeline funnel:

Signal Pack → Candidates → Decision Engine → Content Jobs → Drafts → Rendering → Review → Publishing → Learning

Entity hierarchy

Entity	Description
Project	Named content brand (e.g. SNS). Container for strategy, flows, prompts, learning rules. Stored in <code>caf_core.projects</code> ; URLs use <code>slug</code> .
Run	One execution cycle for a project, tied to a signal pack. Groups all work from a single evidence window.

Signal pack	Research bundle attached to a run. Curated planner rows live in <code>jobs_json</code> (dual-written with legacy <code>ideas_json</code>). Legacy intake also used <code>overall_candidates_json</code> .
Candidate	One idea × one flow type. Built in memory from signal pack rows × enabled flows. Scored by the decision engine. Not persisted as first-class DB rows in current app code (historical <code>caf_core.candidates</code> table exists but is not the planning source of truth).
Content job	Atomic executable unit. Created from planned candidates. Central entity — rendering, review, publishing, and learning key off <code>task_id</code> .
Job draft	One LLM generation attempt for a job (<code>attempt_no</code> , <code>revision_round</code>).
Asset	Produced media (carousel image, video, audio, subtitles) linked to a job.
Editorial review	Human approval / rejection / edit decision on a job.
Diagnostic audit	Machine-generated quality evaluation.
Performance metric	Post-publication outcome data.
Learning rule	Structured insight that changes future behavior (prompt selection, routing, scoring).
Experiment	Controlled intervention to measure impact of changes.
Publication placement	Scheduled or completed publish intent per platform.

Inputs pipeline entities (upstream of signal packs)

Entity	Description
Evidence import	XLSX or scraper ingest → <code>inputs_evidence_imports</code> + <code>inputs_evidence_rows</code> .
Evidence insights	Row-level LLM/vision passes → <code>inputs_evidence_row_insights</code> (broad, top-performer tiers).
Ideas	Structured idea list → <code>ideas</code> , <code>signal_pack_ideas</code> , <code>inputs_ideas</code> .
Brand bible	Versioned Brand Visual System per project → <code>brand_bibles</code> ; snapshot to <code>generation_payload.bvs_v1</code> .
Brand profile	Marketer voice/strategy → <code>brand_profiles</code> .
Creative Intelligence	Archived top-performer media + vision analysis → <code>creative_*</code> tables.

See `docs/CAF_INPUTS_PIPELINE_ROADMAP.md` ,
`docs/CREATIVE_INTELLIGENCE.md` , and
`docs/CAF_CURRENT_STATE_CONTEXT_PACK.md` .

ID conventions

Entity	Field	Pattern	Example
Run	run_id	{PROJECT}_{period}	SNS_2026W0
Candidate	candidate_id	{run_id}_{platform}_{NNNN} OR {base}_{flow_type}	SNS_2026W0
Job	task_id	{run_id}__{platform}__{flow_type}__row{NNNN}__{variation}	SNS_2026W0
Draft	draft_id	d_{random12}	d_mn398eop
Asset	asset_id	{candidate_id}__{ASSET_TYPE}_v{version}	SNS_2026W0
Scene	scene_id	{task_id}__scene_{NN}	...row0012

task_id is the main execution key. Downstream tables (`job_drafts` , `assets` , `editorial_reviews` , `job_state_transitions` , `diagnostic_audits` , `performance_metrics` , `auto_validation_results`) reference it.

Relationships (text joins, not UUID FKs)

- `content_jobs.run_id` = `runs.run_id` (same text key, scoped by `project_id`)
 - `content_jobs.candidate_id` = decision-engine candidate id (text)
 - `job_drafts.task_id` = `content_jobs.task_id`
 - Cross-table joins use **(`project_id`, `task_id`)** or **(`project_id`, `run_id`)**
-

Run lifecycle

```
CREATED → PLANNING → PLANNED → GENERATING → RENDERING → REVIEWING →  
COMPLETED  
  
CANCELLED  
  
↘ FAILED /
```

Planning prerequisite: Materialize planner rows into `runs.planned_jobs_json` (canonical; dual-written with legacy `candidates_json`) via `POST /v1/runs/:project_slug/:run_id/jobs` before `POST .../start` .

Job lifecycle

Typical path:

PLANNED → GENERATING → GENERATED → (QC) → RENDERING → IN_REVIEW →
APPROVED | REJECTED | NEEDS_EDIT

QC may route to `BLOCKED` , `REJECTED` , or `NEEDS_EDIT` . `QC_FAILED` is set only by the limited `POST /v1/pipeline/.../full` path — not the main `job-pipeline.ts` . Video jobs may remain `RENDERING` during HeyGen/Sora polls.

Full detail: `docs/LIFECYCLE.md` .

Editorial decisions

`APPROVED` | `NEEDS_EDIT` | `REJECTED`

Critical JSON contract: `generation_payload`

On `caf_core.content_jobs` . Integration hub for pipeline, Review, and admin.

Slice	Canonical helpers
<code>generated_output</code>	<code>src/domain/generation-payload-output.ts</code>
<code>qc_result</code>	<code>src/domain/generation-payload-qc.ts</code> — write via <code>mergeGenerationPayloadQc</code> only
<code>render_state</code>	<code>src/domain/content-job-render-state.ts</code> — <code>hasActiveProviderSession</code> for HeyGen idempotency
Mimic render truth	<code>mimic_v1</code> ON <code>generation_payload</code> (<code>src/domain/mimic-payload.ts</code>) — includes <code>execution_mode: classic why_mimic new_visual</code>
Brand Visual System	<code>bvs_v1</code> ON <code>generation_payload</code> (<code>src/domain/bvs-v1.ts</code>)
TP-grounded carousel review	<code>mimic_carousel_package</code> — all <code>isTpGroundedCarouselRenderFlow()</code> flows; not <code>carousel_package</code>

Invariants (do not break without explicit approval)

1. `task_id` remains the primary execution key across all job-related tables.

- 2. Do not rename or restructure the ID hierarchy casually.
- 3. Preserve parent-child traceability (scene bundles: `parent_id` / `parent_candidate_id`).
- 4. New tables follow the **text-ID join** pattern.
- 5. Status values stay compatible with existing state machines.
- 6. `content_jobs` is the source of truth for job state.

Flow type families (planning / pipeline)

Family	Examples	Notes
Carousel	FLOW_CAROUSEL	Standard HBS renderer path
Video	FLOW_* video kinds, HeyGen, scene assembly	See flow-kind.ts
Product video	FLOW_PRODUCT_*	Product-focused video flows
Mimic	FLOW_TOP_PERFORMER_MIMIC_IMAGE, FLOW_TOP_PERFORMER_MIMIC_CAROUSEL, FLOW_VISUAL_FIRST_CAROUSEL, FLOW_WHY_MIMIC_CAROUSEL	Requires MIMIC_IMAGE_ENABLED for render; visual-first is new visual (no TP frames)
Image product	FLOW_IMG_*	Registered; not fully wired to generation
Offline	Various	Excluded from pipeline — offline-flow-types.ts

See also

- [EXTERNAL CONTEXT PACK.md](#) — what to upload to external LLMs
- [DATABASE SCHEMA.md](#) — table catalog
- [LIFECYCLE.md](#) — full state machines
- [ARCHITECTURE.md](#) — where code lives

Included: docs/DATABASE_SCHEMA.md

CAF Core — Database schema reference

Purpose: Table catalog for engineers, external re-implementers, and LLMs that need to understand **where state lives** without reading every migration.

Schema name: `caf_core`

Migrations: `migrations/*.sql` (applied via `npm run migrate` or

CAF_RUN_MIGRATIONS_ON_START) — through `078_brand_bibles.sql` and later as added

Tracking table: `caf_core.schema_migrations`

Updated current-state note (2026-07): Prefer

`docs/CAF_CURRENT_STATE_CONTEXT_PACK.md` §5 when this catalog lags new migrations.

Join convention: Prefer `(project_id, task_id)` or `(project_id, run_id)` text keys — not UUID foreign keys across aggregates.

Core pipeline tables

Table	Role
projects	Tenants; slug used in URLs
project_system_constraints	System-wide caps per project
runs	Run header; run_id, status, signal_pack_id, planned_jobs_json / candidates_json, context_snapshot_json, plan_summary_json
signal_packs	Research bundles; jobs_json, ideas_json, overall_candidates_json, platform summaries, visual_guidelines_pack_v1
content_jobs	Central table — task_id, status, generation_payload, render_state, QC columns
job_drafts	LLM attempts per task_id
assets	Rendered media linked to jobs
job_state_transitions	Audit log of status changes
decision_traces	Planning decision snapshots
candidates	Historical — not current planning source of truth

Review, QC, and validation

Table	Role
editorial_reviews	Human decisions (APPROVED, NEEDS_EDIT, REJECTED)
diagnostic_audits	Machine diagnostics on jobs
auto_validation_results	Automated validation records

validation_events	Validation event stream
llm_approval_reviews	Post-approval LLM review scores + upstream_recommendations
run_output_reviews	Run-level output reviews

Flow engine & QC metadata

Table	Role
flow_definitions	Flow types, routing, QC checklist names
prompt_templates	Prompt bodies and versions
output_schemas	JSON schema for LLM output validation
carousel_templates	Carousel HBS template metadata
qc_checklists	QC check rows (required_keys, regex, ...)
risk_policies	Keyword risk policies (scoped by applies_to_flow_type)
prompt_labs_overrides	Prompt lab experiment overrides

Project configuration

Table	Role
strategy_defaults	Strategy JSON per project
brand_constraints	Brand voice, banned words
platform_constraints	Per-platform limits
risk_rules	Project risk rows (not enforced by QC today — see RISK_RULES.md)
allowed_flow_types	Enabled flows per project
heygen_config	HeyGen avatar / voice settings
project_product_profile	Product video profile
project_brand_assets	Brand images (logo, slide frame, style refs) for HeyGen / mimic / BVS
brand_profiles	Marketer voice/strategy profile (migrations/072_brand_profiles.sql)
brand_bibles	Versioned Brand Visual System — brand_bible_v1 JSON; snapshotted to generation_payload.bvs_v1 (078_brand_bibles.sql)
project_carousel_templates	Project-specific carousel templates

project_integrations	Meta FB/IG IDs, tokens metadata
reference_posts, viral_formats	Reference content config

Learning & performance

Table	Role
learning_rules	Active/pending rules (rule_family, action_type, scopes)
suppression_rules	Planning suppression
learning_observations	Evidence observations (incl. upstream rec items)
learning_hypotheses, learning_hypothesis_trials	Experiment tracking
learning_insights	Compiled insights
learning_generation_attribution	Which rules affected a generation
performance_metrics	Post-publish metrics
performance_ingestion_batches	Batch ingest metadata
run_content_outcomes	Run-level outcome summaries
job_outcomes	Per-job publish → performance anchor (migrations/075_job_outcomes.sql)

Publishing

Table	Role
publication_placements	Publish intent + status (draft → published / failed)

Inputs & evidence pipeline

Table	Role
inputs_evidence_imports	XLSX / ingest batch header
inputs_evidence_rows	Normalized evidence rows
inputs_evidence_row_insights	LLM/vision insights per row (tiers: broad, top_performer_*)

inputs_evidence_packs	Packaged evidence for processing
inputs_processing_profiles	Scoring criteria, models, caps
inputs_source_rows	Workbook source registry
inputs_scraper_config, inputs_scraper_runs	Apify scraper config + runs
inputs_idea_lists, inputs_ideas	Idea list intake
ideas, idea_grounding_insights	Structured ideas + grounding
signal_pack_ideas, signal_pack_selected_ideas	Ideas linked to signal packs
insights_packs	Insights pack metadata
qc_flow_profiles	Per-flow QC profile config
evidence_media_assets	Archived media for evidence rows

Creative Intelligence (top-performer mimic upstream)

Table	Role
creative_source_assets	Ingested top-performer media
creative_visual_analyses	Vision analysis per asset
creative_insights	Aggregated creative insights
creative_carousel_mimic_templates	Carousel mimic template library

Operations & audit

Table	Role
api_call_audit	External API call log + cost estimates
prompt_versions	Prompt version history
experiments	A/B prompt experiments
generation_counters	Generation counters per scope

Key columns on content_jobs

Column	Purpose
task_id	Primary execution key (with project_id)
run_id	Parent run (text)
candidate_id	Planner candidate id (text)
flow_type, platform	Routing / QC scope
status	Job lifecycle state
qc_status, recommended_route	QC outcome (also inside payload)
generation_payload	Main JSON contract — prompts, LLM output, QC, mimic_v1, bvs_v1, publish URLs
render_state	Provider session state (HeyGen video_id, etc.)
scene_bundle_state	Scene assembly progress
review_snapshot	Snapshot for review UI

Key columns on runs

Column	Purpose
run_id	Human-readable run key (with project_id)
signal_pack_id	Attached research bundle
status	Run lifecycle
planned_jobs_json	Materialized planner rows (canonical)
candidates_json	Legacy alias (dual-written)
context_snapshot_json	Prompt versions + learning fingerprints at plan time

ER sketch (logical, not SQL FKs)

```
erDiagram
    projects ||--o{ runs : has
    projects ||--o{ signal_packs : has
    runs ||--o| signal_packs : attaches
    runs ||--o{ content_jobs : plans
```

```
content_jobs ||--o{ job_drafts : generates
content_jobs ||--o{ assets : produces
content_jobs ||--o{ editorial_reviews : reviewed
content_jobs ||--o{ publication_placements : publishes
projects ||--o{ learning_rules : learns
```

Bootstrap order for a fresh database

1. `npm run migrate` — applies all `migrations/*.sql` in order
 2. `npm run seed:demo` or `npm run seed:canonical-flows` — optional demo project + flows
 3. `npm run seed:flow-engine` — flow definitions and prompts (as needed)
 4. Configure project via admin or `import:project-csv`
-

See also

- [DOMAIN MODEL.md](#) — IDs and lifecycles
- [REBUILD FROM DOCS.md](#) — full bootstrap guide
- [ARCHITECTURE.md](#) — which repositories read/write these tables
- `migrations/001_caf_core_schema.sql` — original core DDL

Table list aligned with migrations through 069. When in doubt, `grep CREATE TABLE in migrations/` .

Included: AGENTS.md

AGENTS.md — Guidance for AI coding assistants

This file helps **Cursor agents** (and humans) work safely and efficiently in **CAF Core**.

Project identity

- **Repo:** CAF Core — Fastify + PostgreSQL backend for a **content automation pipeline** (signals → jobs → LLM → QC → render → review → publish → learning).
- **Source of truth:** Postgres schema `caf_core` + `content_jobs` rows, especially `generation_payload` .
- **Review app** (`apps/review`) is a **client** of Core APIs, not the authority for job state.

Always read first

Resource	Why
docs/CAF_CURRENT_STATE_CONTEXT_PACK.md	Repo-derived current state — start here when context may be stale (BVS, new visual, Why Mimic)
docs/EXTERNAL_CONTEXT_PACK.md	Tiered doc bundle for ChatGPT / other repos / rebuilds
docs/DOMAIN_MODEL.md	ID conventions , <code>task_id</code> joins, lifecycle states (external copy)
.cursor/rules/caf-domain-model.mdc	Same domain rules (Cursor always-on)
docs/ARCHITECTURE.md	Layers, critical files, QC/learning split
src/config.ts	Env flags (Zod); defaults for validation, human review after QC, URLs

Invariants — do not break without explicit user approval

- `task_id` is the primary execution key across `job_drafts`, `assets`, `editorial_reviews`, `job_state_transitions`, etc., scoped by `project_id`.
- Prefer joins on `(project_id, task_id)` or `(project_id, run_id)` using **text IDs** as documented in `caf-domain-model`.
- `content_jobs.generation_payload` is the main JSON contract; coordinate changes with pipeline, Review, and admin consumers. Use `mergeGenerationPayloadQc` (`src/domain/generation-payload-qc.ts`) to write the `qc_result` slice — it is the only sanctioned writer for that subset.
- HeyGen retry**: avoid double-submitting renders when `render_state` already holds `video_id` / `session_id`. Use `hasActiveProviderSession` from `src/domain/content-job-render-state.ts` — do not re-implement the check inline.
- Learning lookups go through the facade** at `src/services/learning-rule-selection.ts` — `getLearningRulesForPlanning` for planning, `getLearningContextForGeneration` for prompts. Do not import `listActiveAppliedLearningRules` or `compileLearningContexts` directly from new code.
- QC risk enforcement today**: `risk_policies` (scoped per `flow_type` via `applies_to_flow_type`) + `brand_constraints.banned_words`. Project `risk_rules` are **not** enforced by `qc-runtime.ts`; **GET** `/v1/projects/:slug/risk-qc-status` is the canonical honesty endpoint.

7. **Review app /v1/ contract** is covered by `src/routes/review-contract.test.ts` . If you rename or remove a listed path, update that test in the same change.
8. **Typed payload readers** — prefer the helpers in `src/domain/` over ad-hoc casts:
 - `pickGeneratedOutput / hasGeneratedOutput` for `generation_payload.generated_output`
 - `pickRenderState / hasActiveProviderSession / isMidProviderPhase` for `render_state`
 - `pickStoredQcResult / mergeGenerationPayloadQc` for `qc_result`
 - `pickMimicPayload` for `generation_payload.mimic_v1`
 - `parseBvsV1 / attachBvsToPlannedPayload` for `generation_payload.bvs_v1`
 - `pickMimicCarouselDraftPackage` for `mimic_carousel_package` (TP-grounded carousel flows only)
9. **Structured logs**: when adding new pipeline logging, prefer `logPipelineEvent(level, stage, message, { run_id, task_id, job_id, ... })` from `src/services/pipeline-logger.ts` over `console.*` . Correlation fields are how we trace a single job across stages.
10. **Upstream recommendations** (`src/domain/upstream-recommendations.ts`) are the structured shape the post-approval LLM reviewer emits. Writers must use `parseUpstreamRecommendations` (tolerant) and persist via `insertLlmApprovalReview` ; each parsed item is also logged as its own `learning_observation` — do not bypass either when extending the reviewer.
11. **Run context snapshot** (`src/services/run-context-snapshot.ts`) is the canonical record of "what we actually generated with" (prompt versions + brand/strategy slice + learning fingerprints). Persist via `setRunContextSnapshot` . Snapshot failures are logged but must never abort a run.
12. **Mimic carousel package** — `mimic_carousel_package` is for **TP-grounded carousel render flows** (`isTpGroundedCarouselRenderFlow()`): `FLOW_TOP_PERFORMER_MIMIC_CAROUSEL` , `FLOW_VISUAL_FIRST_CAROUSEL` , `FLOW_WHY_MIMIC_CAROUSEL` . Do not conflate with `FLOW_CAROUSEL / carousel_package` . Render truth is `mimic_v1` on `generation_payload` . Mimic LLM prompts must filter signal pack to the job's single planned idea (`llm-creation-pack-budget.ts` , `mimicFlowOnly: true`).
13. **Mimic carousel text overlay** — TP-grounded carousels must **not** bake LLM copy into image models at render. Overlay-only (DocAI/HBS + `reprint-text-overlay`) is

enforced in `job-pipeline.ts` ; do not re-enable Flux typography for carousels without explicit approval.

14. **New visual carousel** — `FLOW_VISUAL_FIRST_CAROUSEL` uses `execution_mode: "new_visual"` on `mimic_v1` : idea + BVS driven, **no** top-performer `reference_items` . Prep in `new-visual-carousel-prep.ts` — not TP reference resolve. Do not treat visual-first as competitor frame replication.
15. **Why Mimic carousel** — `FLOW_WHY_MIMIC_CAROUSEL` uses `execution_mode: "why_mimic"` with Slide Intelligence (`slide_intelligence` on `mimic_v1`). Same render engine as other TP-grounded flows; copy/prompts are SIL-driven.
16. **Brand Visual System (BVS)** — versioned `brand_bibles` (`brand_bible_v1`) snapshot to `generation_payload.bvs_v1` when `candidate_data.use_brand_visual_system` is set (visual-first defaults on). `mimic_v1.bvs_render_plan` drives invented plates for `template_bg` . Use `src/domain/brand-bible.ts` , `bvs-v1.ts` , `bvs-render-plan.ts` — do not ad-hoc read bible JSON from jobs.

Where to change behavior

Goal	Start here
Run planning, scoring, suppression	<code>src/decision_engine/</code> , <code>src/services/run-orchestrator.ts</code>
LLM prompts & creation pack	<code>src/services/llm-generator.ts</code> , <code>src/repositories/flow-engine.ts</code>
QC checks / risk keywords	<code>src/services/qc-runtime.ts</code> , checklist rows, scoped <code>risk_policies</code> via <code>listRiskPoliciesForJob</code>
Job lifecycle & render	<code>src/services/job-pipeline.ts</code>
Top-performer mimic (prep, render, modes)	<code>src/services/mimic-draft-prep.ts</code> , <code>mimic-carousel-render.ts</code> , <code>mimic-image-job.ts</code> , <code>mimic-mode-classifier.ts</code> , <code>src/domain/mimic-payload.ts</code>
New visual carousel (<code>FLOW_VISUAL_FIRST_CAROUSEL</code>)	<code>src/services/new-visual-carousel-prep.ts</code> , <code>new-visual-carousel-execution.ts</code> , <code>new-visual-carousel-flux-prompts.ts</code>
Why Mimic carousel (<code>FLOW_WHY_MIMIC_CAROUSEL</code>)	<code>src/domain/why-mimic-carousel-flow-types.ts</code> , <code>why-mimic-execution.ts</code> , SIL ON <code>mimic_v1</code>
Brand Visual System (BVS)	<code>src/domain/brand-bible.ts</code> , <code>bvs-v1.ts</code> , <code>bvs-render-plan.ts</code> , <code>src/services/bvs-render-overlays.ts</code> , <code>src/repositories/brand-bibles.ts</code>
Mimic text overlay / DocAI / reprint	<code>src/domain/mimic-docai-layer-positions.ts</code> , <code>mimic-post-render-layout-loop.ts</code> , Review <code>reprint-text-overlay</code>

Human review & rework	src/routes/v1.ts, src/services/rework-orchestrator.ts
Publications	src/routes/publications.ts
Learning APIs & compiled guidance	src/routes/learning.ts, facade at src/services/learning-rule-selection.ts
New HTTP routes	Register in src/server.ts (pattern: register*Routes)

Conventions

- **TypeScript ESM** — imports use `.js` extensions in source (Node ESM).
- **Tests:** Vitest — `npm test` ; colocate `*.test.ts` under `src/` .
- **Migrations:** SQL in `migrations/` ; applied via `src/cli/migrate.ts` / startup.
- **Avoid** drive-by refactors in files unrelated to the task; match existing style.

Commands (from repo root)

```
npm run dev          # Core API (watch)
npm run migrate      # Apply DB migrations
npm test             # Vitest
npm run process-run -- <run_id|uuid> [--project SLUG]
```

Review app: `cd apps/review && npm run dev` (needs `CAF_CORE_URL`).

Production Review / Admin UI: embedded in the Core Fly image — **not** a separate Vercel deploy.
 Canonical URL: <https://caf-core.fly.dev/admin/workbench>. After `apps/review/` changes, ship with `fly deploy -a caf-core` from repo root (rebuilds Next.js standalone in `Dockerfile`). Vercel projects (`caf-core-review` , etc.) are optional/legacy; do not assume they are what operators use.

Documentation map

Doc	Audience
docs/CAF_PRODUCT_PITCH.md	Leadership / investors / evaluators
docs/CAF_COMPLETE_PRODUCT_GUIDE.md	Complete product guide — what CAF is and does
docs/CAF_CURRENT_STATE_CONTEXT_PACK.md	Current repo truth — operational map; PDF volumes in docs/export/pdf/11-14

docs/EXTERNAL_CONTEXT_PACK.md	ChatGPT / external repos — what to upload, system prompt
docs/REBUILD_FROM_DOCS.md	Engineers bootstrapping from scratch
docs/DATABASE_SCHEMA.md	Postgres table catalog
docs/DOMAIN_MODEL.md	Entities, IDs, lifecycles
README.md	Quick start, API index, deploy
docs/CAF_CORE_COMPLETE_GUIDE.md	Single-file full project reference
docs/PROJECT_OVERVIEW.md	Stakeholders / onboarding
docs/ARCHITECTURE.md	Engineers
docs/LIFECYCLE.md	Run & job states
docs/TECH_STACK.md	Stack & services
docs/layers/README.md	Per-layer deep dives
docs/QUALITY_CHECKS.md	QC runtime
docs/GENERATION_GUIDANCE.md	Prompt guidance
docs/RISK_RULES.md	Risk policies vs project risk_rules
docs/MIMIC_FLOWS_COMPLETE_GUIDE.md	Top-performer mimic (full; lags new visual / BVS — prefer current-state pack)
docs/MIMIC_IMAGE_FLOWS.md	Mimic image quick ref
docs/MIMIC_TEXT_PLACEMENT_AUTOMATION.md	Future mimic text placement QA (post-render composite loop) — read before automating
docs/CREATIVE_INTELLIGENCE.md	Top-performer ingest upstream of mimic
docs/API_REFERENCE.md	HTTP examples

Optional rules (Cursor)

- scene-assembly-n8n-legacy.mdc — when touching scene/n8n-legacy paths
- mimic-signal-pack-llm-filter.mdc — mimic LLM creation pack; single-idea signal_pack filter
- mimic-carousel-package.mdc — mimic_carousel_package vs FLOW_CAROUSEL / carousel_package
- visual-first-carousel-flow.mdc — FLOW_VISUAL_FIRST_CAROUSEL : **new visual** lane (execution_mode:

"new_visual" , idea+BVS, no TP references); same mimic **render engine** as manual mimic; Review workbench without original-vs-generated compare

- `mimic-text-placement-automation.mdc` — **before automating mimic text placement**: post-render composite QA loop, HTML overlay invariants, `docai_layer_positions` schema
- `caf-domain-model.mdc` — **alwaysApply**

If the user's request conflicts with **domain invariants**, **ask** or **surface the tradeoff** before renaming IDs or changing status enums.